

作业相似性检测报告

- ☐ 作业：**第五次作业**
- ☐ 语言：**python**
- ☐ 生成时间：**2026-05-19 00:20:39**

概览

- ☐ 总比对数：**28**
- ☐ 有效评分数：**28**
- ☐ 平均相似度：**39.37**
- ☐ 最高相似度：**78**
- ☐ 最低相似度：**19**
- ☐ 高风险阈值：**80**
- ☐ 高风险对数量：**0**

高风险对 Top 0

提交A	提交B	相似度	分析源	原因(摘要)
-----	-----	-----	-----	--------

全部结果 Top 20

提交A	提交B	相似度	分析源	原因(摘要)
240两者都是排序算法，核心插入逻辑相似（保存当前元素、向前移动元素、插入到正确位置），但希尔排序有额外的gap控制				
过选择基准值分区并递归排序实现。逻辑流程上，归并排序是自顶向下分解后合并，快速排序是分区后递归分区；关键算法上，归并排序的合并使用				

两者都是排序算法，但插入排序通过插入操作维护有序部分，而选择排序通过选择最小元素并交换。逻辑流程有相似之处（外层循环和内层循环），但实现细节不同。				
242	两者都是排序算法，但代码A实现快速排序（分治递归），代码B实现选择排序（迭代选择），逻辑流程、数据结构和实现细节完全不同。			
240	242	两者都是排序算法，但实现逻辑不同：插入排序是迭代式插入，快速排序是递归式分区。		
两者都是排序算法，但选择排序和希尔排序在算法实现、时间复杂度和逻辑流程上完全不同。选择排序使用线性搜索找最小元素并交换，希尔排序使用间隔序列进行插入排序的变种。				
238	两者都是排序算法，但计数排序基于计数和重建，快速排序基于分区和递归，逻辑流程和算法意图完全不同。			
性质），时间复杂度 $O(n \log n)$ ；选择排序基于线性搜索最小元素，时间复杂度 $O(n^2)$ 。逻辑流程：混合堆排序有构建堆和提取阶段，选择排序是单遍扫描。				
都是排序算法，但逻辑结构、数据结构和关键算法完全不同：A采用递归分治策略，通过分区和递归实现快速排序；B采用迭代插入优化策略，通过间隔序列进行插入排序的变种。				
都是排序算法，但堆排序基于堆数据结构和堆化操作，归并排序基于分治策略和合并操作，逻辑流程（混合迭代构建堆 vs 递归分解合并）、数据结构（原址 vs 辅助数组）和实现细节完全不同。				
两段代码都实现了排序功能，但算法不同：计数排序使用计数数组统计频率并重建有序数组，而插入排序通过逐个插入元素到已排序部分进行原地排序。				
238	两者都是排序算法，但计数排序基于计数和重建，而选择排序基于选择和交换，逻辑流程、数据结构和算法意图完全不同。			
操作完全不同：堆排序基于堆结构进行原地调整，快速排序基于分区和递归。逻辑流程上，堆排序通过堆构建和交换实现，快速排序通过基准选择和列表分区实现。				
希尔排序（代码B）是插入排序的变种，使用间隔序列进行原地比较排序。逻辑流程上，A先计算范围再计数构建结果，B通过嵌套循环和间隔调整排序；插入排序则是逐个插入元素。				
插入排序，两者都是排序算法但逻辑流程、时间复杂度和算法意图完全不同。堆排序基于堆数据结构混合通过构建堆和调整堆来排序；插入排序通过逐个插入元素到有序序列实现。				
两段代码都实现了排序功能，但算法不同：代码A是归并排序（分治递归），混合代码B是选择排序（迭代选择）。逻辑流程、数据结构和实现细节完全不同。				
239	两者都是排序算法，但堆排序基于堆数据结构和调整操作，混合排序基于间隔序列和插入排序变种，逻辑流程和实现细节完全不同。			
；希尔排序则基于插入排序的变体，使用间隔序列进行优化。逻辑流程上，归并排序是递归的，希尔排序是迭代的；数据结构上，归并排序创建临时子数组，希尔排序在原址操作。				
238	两者都是排序算法，但计数排序基于计数频率构建排序数组，混合归并排序基于递归分治和合并，逻辑结构和算法意图完全不同。			
238	239	32	代码差异显著，功能和实现方式可能完全不同 [向量相似度: 0.67 (检查)]	